

Summer school ISIMA - OTH Phase 2

Mathematical Modeling and Optimization with Julia

Prof. Dr. Viet Hung Nguyen - ISIMA - Clermont Auvergne INP

Lesson 1 - Fundamentals of Julia language

August 2026

Summary of the course

- Fundamentals of Julia language
- Introduction to JuMP and Linear Programming models
- Non Linear Programming models
- Integer Programming models
- Modeling of Uncertainty, Fairness and Multicriteria in Optimization
- Practical Sessions

Evaluation: Project to be done by teams of 2 (or exceptionally 3) persons. The last lecture will be devoted to the defense of the project. You must be able to explain every detail of your program.

The birth of Julia

- Created in 2009 by Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah in MIT
- Publicly announced in 2012
- Designed for scientific computing, numerical optimization, and data analysis
- Main goal: combine Python's ease of use with C-like performance

The two-language problem

Julia aims to eliminate the need to prototype algorithms in a high-level language and rewrite them in a low-level language to obtain high performance.

Main features and comparisons with C and Python

Feature	Julia	Python	C
Execution	JIT-compiled	Interpreted	Compiled
Performance	Close to C	Usually slower	Very high
Syntax	High-level	High-level	Low-level
Typing	Dynamic	Dynamic	Static
Memory	Automatic	Automatic	Mostly manual
Multiple dispatch	Native	Limited	No
Interactive use	Excellent	Excellent	Limited

Julia for Scientific Computing

- Near-C performance with high-level, readable syntax
- Efficient numerical loops without mandatory vectorization
- Native support for matrices, complex numbers, and mathematical notation
- Multiple dispatch for generic and reusable algorithms
- Built-in support for parallel and distributed computing
- Rich scientific ecosystem:
 - `LinearAlgebra` for linear algebra
 - `DifferentialEquations.jl` for differential equations
 - `Plots.jl` for data visualization

Main strength

Julia allows scientists to write readable mathematical code without sacrificing computational performance.

Julia for Mathematical Optimization

- JuMP: an expressive algebraic modeling language
- Support for a wide range of optimization problems:
 - linear and mixed-integer optimization
 - nonlinear and conic optimization
 - stochastic and multiobjective optimization
- Common interface to many solvers: HiGHS, Gurobi, CPLEX, Ipopt, and SCIP
- Easy access to variables, constraints, dual values, and solver statistics
- Models and specialized algorithms can be implemented in the same high-performance language

Main strength

Julia combines mathematical modeling, algorithm development, and high-performance implementation.

Julia and JuMP: Support and Resources

- **Official websites**

- Julia language: <https://julialang.org>
- JuMP: <https://jump.dev>

- **Documentation and tutorials**

- Julia manual and learning resources: <https://docs.julialang.org>
- JuMP tutorials with executable examples:
<https://jump.dev/JuMP.jl/stable/tutorials>

- **Community support**

- Julia Discourse forum: <https://discourse.julialang.org>
- Questions and examples on Stack Overflow
- Active open-source development on GitHub

- **Cross-platform availability**

- Available on Windows, macOS, and Linux
- Consistent package management across platforms
- Usable through Pluto, Jupyter, and the command line

A mature and accessible ecosystem

Julia and JuMP provide extensive documentation, practical tutorials, and active community support for both beginners and advanced users.

Installing Julia

- 1 Go to <https://julialang.org/downloads>
- 2 Download the installer for:
 - Windows
 - macOS
 - Linux
- 3 Follow the installation instructions
- 4 Start Julia to open the interactive prompt

Cross-platform installation

Julia uses the same language and package manager on Windows, macOS, and Linux.

Installing Essential Packages

From the Julia prompt, enter:

```
using Pkg
```

```
Pkg.add("Pluto")
```

```
Pkg.add("Plots")
```

```
Pkg.add("JuMP")
```

```
Pkg.add("HiGHS")
```

Pluto Interactive notebooks

Plots Data visualization

JuMP Optimization modeling

HiGHS Open-source optimization solver

Important

A package is installed only once, but must be loaded with `using` in each new Julia session.

- **Command line**

- Julia's interactive prompt (REPL)
- Convenient for testing short instructions

- **Integrated development environments**

- Visual Studio Code with the Julia extension
- Recommended for developing larger projects

- **Interactive notebooks**

- Jupyter notebooks through IJulia.jl
- **Pluto.jl**: reactive notebooks designed for Julia

Why Pluto?

Pluto automatically updates dependent cells when a variable changes. It is particularly suitable for interactive teaching, experimentation, and reproducible scientific computing.

The Julia REPL and Its Different Modes

The **REPL** allows instructions to be executed interactively.

Prompt	Mode	Activation	Example
julia>	Julia	Default	<code>sqrt(16)</code>
help?>	Help	Type ?	<code>println</code>
shell>	Shell	Type ;	<code>pwd</code>
pkg>	Package	Type]	<code>add Plots</code>

- Press Enter to execute an instruction.
- Press Backspace on an empty line to return to Julia mode.
- Press Ctrl+D to leave the REPL.

Interactive workflow

The REPL is useful for testing expressions, consulting documentation, managing packages, and running system commands.

Examples of Julia Instructions in REPL Mode

Calculations and variables

```
julia> 2 + 3 * 4  
14
```

```
julia> x = 5  
5
```

```
julia> x^2 + 2*x + 1  
36
```

Built-in functions

```
julia> sqrt(16)  
4.0
```

```
julia> sin(pi / 2)  
1.0
```

Vectors and matrices

```
julia> v = [1, 2, 3]  
3-element Vector{Int64}
```

```
julia> sum(v)  
6
```

```
julia> A = [1 2; 3 4]  
2x2 Matrix{Int64}
```

Loading a package

```
julia> using LinearAlgebra
```

```
julia> det(A)  
-2.0
```

Useful Julia REPL Shortcuts

Shortcut	Action
Up / Down	Browse the command history
Tab	Complete a command or variable name
Ctrl+C	Interrupt the current computation
Ctrl+L	Clear the REPL screen
Ctrl+D	Exit Julia
?	Enter Help mode
]	Enter Package mode
;	Enter Shell mode

Returning to Julia mode

Press Backspace on an empty line to leave Help, Package, or Shell mode.

Main Structure of a Julia Program

```
# 1. Load packages
using Statistics

# 2. Define constants
const SCALE = 2.0

# 3. Define functions
function transform(data)
    return SCALE .* data
end

# 4. Main program
function main()
    data = [1, 2, 3]
    result = transform(data)
    println("Result: ", result)
end

# 5. Start the program
main()
```

A Julia program usually contains:

- 1 package imports;
- 2 constants and data;
- 3 function definitions;
- 4 a main function;
- 5 a call to the main function.

Good practice

Place computations inside functions to improve code organization, reusability, and performance.

Executing a Julia Program from the REPL

Suppose that the program is stored in `test.jl`.

Execute the complete file:

```
julia> include("test.jl")
```

Execute a file located in another directory:

```
julia> include("/path/to/test.jl")
```

Check or change the current directory:

```
julia> pwd()  
julia> cd("/path/to/project")  
julia> include("test.jl")
```

Important

`include` executes the file in the current Julia session: the defined variables and functions remain available in the REPL.

Installing Julia in Visual Studio Code

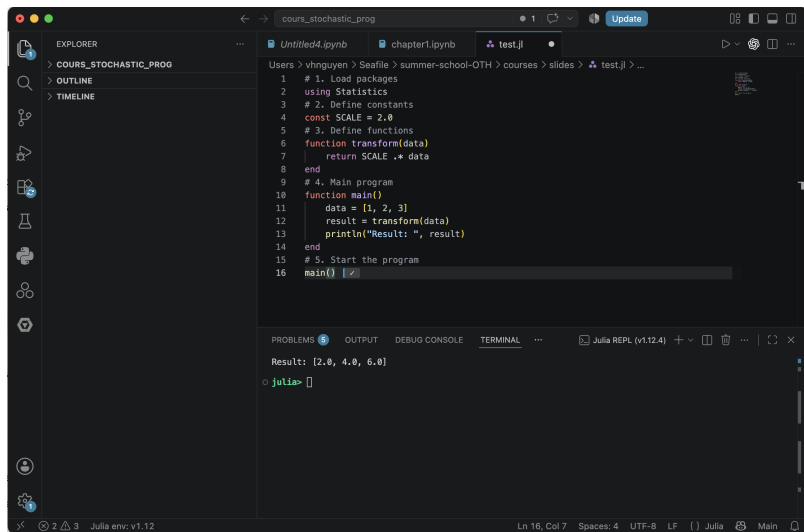
- 1 Install Julia from <https://julialang.org/downloads>
- 2 Install Visual Studio Code from <https://code.visualstudio.com>
- 3 Open the **Extensions** panel in VS Code:
 - press Ctrl+Shift+X;
 - search for Julia;
 - install the official Julia extension.
- 4 Create or open a file with the extension `.jl`
- 5 Start the Julia REPL from the Command Palette:

Julia: Start REPL

Running Julia code

Select some code and press Shift+Enter to execute it in the integrated Julia REPL.

Running Julia in Visual Studio Code



The screenshot shows the Visual Studio Code interface with a Julia script named `test.jl` open. The script contains the following code:

```
1 # 1. Load packages
2 using Statistics
3 # 2. Define constants
4 const SCALE = 2.0
5 # 3. Define functions
6 function transform(data)
7     return SCALE .* data
8 end
9 # 4. Main program
10 function main()
11     data = [1, 2, 3]
12     result = transform(data)
13     println("Result: ", result)
14 end
15 # 5. Start the program
16 main()
```

The `main()` function is highlighted, and the `TERMINAL` panel at the bottom shows the output of the execution:

```
Result: [2.0, 4.0, 6.0]
julia> 
```

The status bar at the bottom indicates the current environment is `Julia env: v1.12` and the cursor is at `Ln 16, Col 7`.

The Julia extension provides an integrated REPL. Select an instruction and press **Shift+Enter** to execute it.

Installing and Starting Pluto

1. Install Pluto from the Julia REPL

```
julia> using Pkg  
julia> Pkg.add("Pluto")
```

2. Start Pluto

```
julia> using Pluto  
julia> Pluto.run()
```

Pluto opens automatically in the default web browser.

Next sessions

Pluto is installed only once. To start it again, simply use:

```
using Pluto    Pluto.run()
```

Using a Pluto Notebook

- 1 Create a new notebook or open an existing `.jl` notebook.
- 2 Enter Julia instructions in a cell.
- 3 Click the **Run** button or press `Shift+Enter`.
- 4 Add new cells using the `+` buttons.
- 5 Save the notebook as a Julia `.jl` file.

Reactive execution

When a variable is modified, Pluto automatically re-executes all dependent cells and updates their results.

Implementing the Program in Pluto

Cell 1: Load the package

```
using Statistics
```

Cell 2: Define the parameter

```
scale = 2.0
```

Cell 3: Define the function

```
function transform(data)
    return scale .* data
end
```

Cell 4: Perform the computation

```
data = [1, 2, 3]
result = transform(data)
```

Cell 5: Display the result

```
println("Result: ",
        result)
```

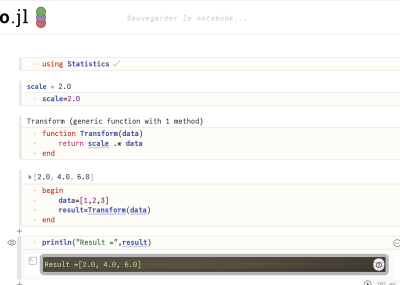
Reactive execution

If scale or data is modified, Pluto automatically re-executes the dependent cells and updates result.

Pluto principle

Each global variable should normally be defined in only one cell.

The Pluto Notebook Interface



```
Pluto.jl  Sauvegarder le notebook...

using Statistics ✓

scale = 2.0
scale=2.0

Transform (generic function with 1 method)
function Transform(data)
    return scale .* data
end

[2.0, 4.0, 6.0]
begin
    data=[1,2,3]
    result=Transform(data)
end

println("Result =",result)
Result =[2.0, 4.0, 6.0]
```

- Julia instructions are entered into interactive cells.
- Results appear immediately below each cell.
- The + buttons create new cells.
- Cells are automatically re-evaluated when their dependencies change.

Reactive execution

Changing one variable automatically updates all dependent results.

Defining Functions in Julia

The Fibonacci sequence is defined by

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2}.$$

Iterative version

```
function fibonacci_iter(n)
    n >= 0 ||
        throw(ArgumentError(
            "n must be
nonnegative"
        ))

    a, b = 0, 1

    for _ in 1:n
        a, b = b, a + b
    end

    return a
end
```

Recursive version

```
function fibonacci_rec(n)
    n >= 0 ||
        throw(ArgumentError(
            "n must be
nonnegative"
        ))

    if n <= 1
        return n
    end

    return fibonacci_rec(n - 1)
        +
            fibonacci_rec(n - 2)
end
```

Types of Function Parameters in Julia

Parameter type	Example
Positional	<code>f(x)</code>
Typed	<code>f(x::Vector{Int})</code>
Optional positional	<code>f(x, order=:ascending)</code>
Keyword	<code>f(x; order=:ascending)</code>
Variable number	<code>f(x...)</code>

```
function sort_sequence( values::Vector{Int}; asc::Bool = true)
    if asc
        return sort(values)
    else
        return sort(values, rev = true)
    end
end
```

Function signature

`values` is a typed positional parameter, whereas `asc` is a typed keyword parameter with a default value.

Calling the Sorting Function

Use the default value

```
values = [7, 2, 9, 1, 5]
sort_sequence(values)
# [1, 2, 5, 7, 9]
```

Specify the keyword parameter

```
sort_sequence(values; asc = false)
# [9, 7, 5, 2, 1]
```

Invalid parameter types

```
sort_sequence([2.5, 1.5])
# MethodError
```

```
sort_sequence(values; asc = 1)
# TypeError
```

Role of the semicolon

Parameters after ; are passed by name. Their order therefore does not matter.

Common Built-in Types in Julia

Julia automatically determines the type of a value.

Category	Julia type	Example
Integer	Int64	<code>x = 42</code>
Floating-point number	Float64	<code>x = 3.14</code>
Boolean	Bool	<code>x = true</code>
Character	Char	<code>x = 'A'</code>
Character string	String	<code>x = "Julia"</code>
Complex number	Complex	<code>x = 2 + 3im</code>
Rational number	Rational	<code>x = 2 // 3</code>

```
typeof(42)           # Int64
```

```
typeof(3.14)         # Float64
```

```
n::Int = 10          y = Float64(n)           # Convert Int to Float64
```

Dynamic typing

A variable does not have a fixed type, but every Julia value has a well-defined concrete type.

Numerical Types and Result Types

The type of an arithmetic result depends on the operand types and the operation.

Expression	Result	Result type
5 + 2	7	Int64
5 + 2.0	7.0	Float64
5 / 2	2.5	Float64
div(5,2)	2	Int64
5 % 2	1	Int64

Inspecting and converting types

```
typeof(5 / 2)           # Float64
typeof(5 // 2)          # Rational{Int64}
Float64(5)              # 5.0
Int(3.8)                # InexactError
Int(round(3.8))          # 4
```

When different numerical types are combined, Julia generally promotes them to a common type that can represent both operands.

Creating and Combining Strings

```
language = "Julia"  
domain = "scientific computing"
```

Concatenation

```
text = language * " for " * domain  
# "Julia for scientific computing"
```

String interpolation

```
name = "Alice"  
score = 18  
  
message = "$name obtained $score points"  
# "Alice obtained 18 points"
```

Repeating a string

```
repeat("Julia! ", 3)  
# "Julia! Julia! Julia! "
```

Main operators

Use `*` for concatenation and `$` to insert values into a string.

Common String Operations

```
s = "Julia programming"
length(s)           # 17
uppercase(s)        # "JULIA PROGRAMMING"
lowercase(s)        # "julia programming"
startswith(s, "Julia") # true
endswith(s, "ing")   # true
occursin("gram", s)  # true
```

Replacing text

```
replace(s, "Julia" => "Scientific")
# "Scientific programming"
```

Splitting and joining

```
words = split(s)
# ["Julia", "programming"]

join(words, "-")
# "Julia-programming"
```

A vector is a one-dimensional array

Arrays are ordered collections. A vector is an array with one dimension.

```
countries = ["Austria", "France", "Belgium"]  
primeNumbers = [2, 3, 5, 7, 11]  
mixed = [2, 3, 5, "seven", 11]
```

Julia uses **1-based indexing**: the first element of a vector has index 1, not 0.

A homogeneous literal such as `[2, 3, 5]` typically has a concrete element type, whereas mixed values may lead Julia to choose a more general element type.

Create an uninitialized typed vector

When the element type and size are already known, a vector can be allocated without initializing its entries.

```
v = Array{Int64}(undef, 5)
```

Equivalent, more explicit one-dimensional form:

```
v = Vector{Int64}(undef, 5)
```

`undef` means that the entries have not yet been initialized. They should be assigned before being used.

Creating Vectors in Julia

Vector creation

```
u = [3,1,0]
```

```
v = [0,3,1]
```

```
zeros(3)      # [0, 0, 0]
```

```
ones(3)       # [1, 1, 1]
```

```
collect(1:5)  # [1, 2, 3, 4, 5]
```

Accessing vector elements

```
u[1]          # 3
```

```
u[2:3]        # [1, 0]
```

```
length(u)     # 3
```

Basic operations

```
u + v         # [3, 4, 1]
```

```
u - v         # [3, -2, -1]
```

```
2 * u         # [6, 2, 0]
```

```
norm(u)       # sqrt(10)
```

Indexing

Julia uses one-based indexing: the first element of `u` is accessed with `u[1]`.

Products of Vectors in Julia

Let $u = [3, 1, 0]^\top$ and $v = [0, 3, 1]^\top$.

Elementwise product

```
u .* v  
# [0, 3, 0]
```

Dot product

$$u^\top v = 3(0) + 1(3) + 0(1) = 3$$

```
dot(u, v)      # 3  
u' * v         # 3
```

Cross product

$$u \times v = \begin{bmatrix} 1 \\ -3 \\ 9 \end{bmatrix}$$

```
cross(u, v)  
# [1, -3, 9]
```

Outer product

```
u * v'  
# 3 x 3 matrix
```

$$uv^\top = \begin{bmatrix} 0 & 9 & 3 \\ 0 & 3 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

Plotting Vector Operations

```
using LinearAlgebra
using Plots
```

```
u = [3.0, 0, 1.0]
v = [0, 3.0, 1.0]
```

```
vectors = [u, v, u + v, cross(u, v)]
names = ["u", "v", "u + v", "u x v"]
colors = [:blue, :red, :green, :purple]
```

```
p = plot3d(
    xlims=(-4, 10), ylims=(-1, 10),
    zlims=(-4, 10),
    xlabel="x", ylabel="y", zlabel="z",
    legend=:topleft
)
```

The four vectors start at the origin:

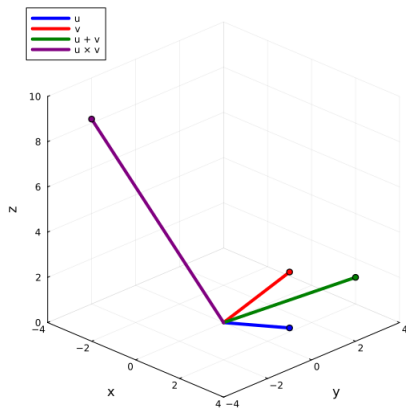
$$u = \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix}, \quad v = \begin{bmatrix} 0 \\ 3 \\ 1 \end{bmatrix},$$

$$u+v = \begin{bmatrix} 3 \\ 3 \\ 2 \end{bmatrix}, \quad u \times v = \begin{bmatrix} -3 \\ 6 \\ -3 \end{bmatrix}.$$

Geometric property

The cross product $u \times v$ is perpendicular to both u and v .

Visualization of Vector Operations



$$u = \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix}, \quad v = \begin{bmatrix} 0 \\ 3 \\ 1 \end{bmatrix}$$

$$u + v = \begin{bmatrix} 3 \\ 3 \\ 2 \end{bmatrix}$$

$$u \times v = \begin{bmatrix} -3 \\ -3 \\ 9 \end{bmatrix}$$

A matrix is a two-dimensional array

Spaces separate columns and semicolons separate rows in a matrix literal.

```
A = [1 2; 3 4]
T = ['a' 'b' 'c';
     'x' 'y' 'z']
```

Thus A is a 2×2 matrix and T is a 2×3 matrix.

More generally, Julia's Array type can represent data with any number of dimensions.

An array of arrays is different from a matrix

A collection can contain arrays as its elements. This is useful for irregular or “jagged” data.

```
A = [[1, 2], [3, 4, 6], [1]]
```

```
A[1]          # [1, 2]
```

```
A[2][3]       # 6
```

Here A is a vector whose elements are vectors of different lengths. It is **not** a rectangular matrix.

For rectangular numerical data, a true matrix is usually more natural and more efficient.

Allocate typed multidimensional arrays

Specify the element type followed by the dimensions.

```
A = Array{Int64}(undef, 2, 3)
B = Matrix{Int64}(undef, 2, 3)
```

Initialized multidimensional arrays are just as easy to create:

```
Z = zeros(Float64, 2, 3, 4)
O = ones(Float64, 2, 3, 4)
P = fill(pi, 2, 3)
```

`Matrix{T}` is an alias for a two-dimensional `Array{T,2}`.

Index multidimensional arrays by dimension

For an ordinary multidimensional array, provide one index per dimension.

`m[2, 1]`

`T[:, 2]`

The colon `:` selects all valid indices in that dimension, so `T[:, 2]` selects the entire second column.

For an array of arrays, indexing happens successively:

`A[2][3]`

Inspect array dimensions with size and length

```
size(T)           # dimensions as a tuple  
size(T, 2)        # size of the second dimension  
length(T)         # total number of elements
```

For a 2×3 matrix:

`size(T) = (2,3),` `size(T,2) = 3,` `length(T) = 6.`

Commas and spaces produce different shapes

These two literals contain the same numbers but have different dimensions:

```
v = [2, 3, 5, 7]    # Vector: 4 elements  
r = [2 3 5 7]       # Matrix: 1 x 4
```

Use `reshape` to reinterpret the same elements with a new shape:

```
reshape(r, 4)  
reshape(r, (2, 2))
```

The total number of elements must remain unchanged.

Assignment does not copy a mutable array

Consider:

```
g = [2, 3, 4, 5]
```

```
h = g
```

```
g[2] = 0
```

```
println(h)           # [2, 0, 4, 5]
```

`g` and `h` refer to the **same mutable array**. Mutating it through one variable is therefore visible through the other.

This aliasing behavior is important whenever arrays are passed around a program.

Choose the appropriate copying operation

Julia provides several related operations:

Operation	Purpose
<code>copy(A)</code>	create a shallow copy of A
<code>deepcopy(A)</code>	recursively copy nested mutable objects
<code>similar(A)</code>	allocate an uninitialized array with similar type and shape

```
B = copy(A)
```

```
C = deepcopy(A)
```

```
D = similar(A)
```

Comprehensions construct arrays concisely

A comprehension describes which expression to evaluate over one or more iterators.

```
[i for i in 1:5]  
[i + j for i in 1:2 for j in 1:4]  
[i + j for i in 1:2, j in 1:4]  
[i for i in 1:10 if i % 2 == 1]  
[exp(i) for i in 1:3]
```

Note the distinction: repeated for clauses produce a one-dimensional result, while comma-separated iterators produce a multidimensional array.

Use vectors as dynamic lists

Let $L = [1,2,3,4,5]$. Elements can be inserted at either end.

```
push!(L, 10)           # add at the end  
pushfirst!(L, 0)       # add at the front
```

`splice!` can replace or insert at a specified position:

```
splice!(L, 3, [L[3], 7, 4])
```

Because these operations mutate L , their names end in `!`.

Remove elements from a vector

Starting again with `L = [1,2,3,4,5]`:

```
pop!(L)           # remove and return the last element
popfirst!(L)      # remove and return the first element
splice!(L, 2)     # remove and return the element at index 2
```

These operations change both the contents and the length of the vector.

Matrices and Matrix Operations in Julia

Creating matrices

`using LinearAlgebra`

```
A = [1 2; 3 4]
```

```
B = [5 6; 7 8]
```

Basic operations

```
A + B
```

```
2 * A
```

```
A * B           # Matrix product
```

```
A .* B          # Elementwise
```

product

```
A .+ 2          # Add 2 to each
```

entry

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}, \quad A.*B = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

Common matrix operations

```
transpose(A)    # Transpose
```

```
A'              # Adjoint
```

```
det(A)          # Determinant
```

```
tr(A)           # Trace
```

```
rank(A)         # Rank
```

```
inv(A)          # Inverse
```

Solving a linear system

```
b = [5, 11]
```

```
x = A \ b       # Solve A*x = b
```

```
# x = [1.0, 2.0]
```

Tuples are ordered and immutable

A tuple is an ordered collection whose structure cannot be modified after creation.

```
countries = ("Austria", "France", "Belgium")  
values = (2, 3, 5, "seven", 11)
```

```
values[1]          # 2  
values[end]        # 11
```

Tuples use parentheses rather than square brackets and, like arrays, are indexed from 1.

A tuple cannot be changed in place

Compare a mutable vector with an immutable tuple:

```
vct = [2, 7]  
push!(vct, 3)      # valid
```

```
tpl = (2, 7)  
push!(tpl, 3)      # ERROR
```

Immutability makes tuples useful for small fixed collections, such as coordinates, dimensions, or multiple return values.

Dictionaries map keys to values

A dictionary stores key-value associations instead of positions alone.

```
code = Dict(  
    43 => "Austria",  
    32 => "Belgium",  
    33 => "France"  
)
```

```
code[33]           # "France"
```

Keys are unique. Dictionary iteration order should generally not be treated as part of the logical meaning of the data.

Dictionaries are mutable

Start with an empty dictionary, then add or remove associations.

```
code = Dict()
```

```
code[49] = "Germany"
```

```
code[33] = "France"
```

```
pop!(code, 33)
```

The key is used to access the associated value; it does not represent a numeric position in the collection.

Iterate over dictionary keys or values

Julia exposes iterators over the keys and values of a dictionary.

```
keys(code)  
values(code)
```

They can be consumed directly by a loop or materialized when an actual array is required:

```
collect(keys(code))  
collect(values(code))
```

Sets store unique values

A Set represents a collection in which each value occurs at most once.

```
a = Set()
```

```
b = Set([1, 2, 2, 3, 4])
```

```
b                                # contains 1, 2, 3, 4
```

```
push!(b, 5)
```

```
delete!(b, 2)
```

Julia sets are **mutable**. Their iteration order should not be used to represent an ordered sequence.

Set-like operations express relationships between collections

```
union([1, 2], [3, 4])  
intersect([1, 2, 3], [3, 4, 5])  
setdiff([1, 2, 3], [3, 4, 5])
```

These operations correspond to familiar mathematical ideas:

- `union`: elements appearing in either collection;
- `intersect`: elements common to both collections;
- `setdiff`: elements of the first collection not in the second.

Choose a data structure according to its semantics

Structure	Ordered	Mutable	Main idea
Array	yes	yes	indexed sequence / grid
Tuple	yes	no	fixed ordered collection
Dictionary	not by semantics	yes	key-value mapping
Set	no	yes	unique elements

Choose the structure that best expresses what the data *means*, not merely one that can store the values.

Exercise 1 : Testing and Visualizing Whether Two Vectors Are Collinear

Two vectors u and v are collinear if there exists $\lambda \in \mathbb{R}$ such that $u = \lambda v$.

Examples

```
are_collinear([1, 2, 3], [2, 4, 6]) # 1
are_collinear([1, 2, 3], [2, 4, 7]) # 0
are_collinear([1, 2], [-3, -6]) # 1
```

Numerical tolerance

For floating-point vectors, a tolerance is preferable to an exact equality test because computations may involve rounding errors.

Exercise 2 : Solving a square linear system $Ax=b$

Condition	Solutions
$\text{rank}(A) = n$	Unique solution $x=A \setminus b$
$\text{rank}(A) = \text{rank}([A \ b]) < n$	Infinitely many solutions, e.g $x=\text{pinv}(A)*b$
$\text{rank}(A) < \text{rank}([A \ b])$	No exact solution, least square approximation

- If A is invertible, $A \setminus b$ computes the unique solution.
- If the system has infinitely many solutions, $\text{pinv}(A) * b$ returns the solution with minimum Euclidean norm.
- If the system is inconsistent, the function returns the approximate least square solution $\text{pinv}(A) * b$.

Calling the function

```
result = solve_system(A, b)
```

```
println(result.status)
```

```
println(result.x)
```


A short introduction to Graphs.jl and DataStructures.jl

We aim at learning to

- create and inspect directed and undirected graphs in Julia;
- represent weights separately from the graph topology;
- use a heap or priority queue from `DataStructures.jl`;
- implement Dijkstra's algorithm for nonnegative arc lengths;
- reconstruct a shortest path from predecessor information.

Why these two packages?

Graphs.jl

- graph topology: vertices, edges, adjacency lists;
- basic operations: add edges, iterate over neighbors;
- classical graph algorithms are also available.

DataStructures.jl

- heaps, priority queues, stacks, queues, dictionaries with defaults;
- useful when the algorithm needs a specific data structure;
- for Dijkstra: repeatedly extract the vertex with smallest tentative distance.

General principle

Use `Graphs.jl` for the network and `DataStructures.jl` for the algorithmic data structure.

Installing and loading the packages

```
using Pkg  
Pkg.add("Graphs")  
Pkg.add("DataStructures")
```

In a Julia program or notebook:

```
using Graphs  
using DataStructures
```

For reproducible teaching material, prefer a project environment:

```
using Pkg  
Pkg.activate(".")  
Pkg.add(["Graphs", "DataStructures"])
```

Graphs.jl: first example

```
using Graphs
```

```
Undirected graph with 5  
vertices
```

```
= SimpleGraph(5)
```

```
add_edge!(G, 1, 2)
```

```
add_edge!(G, 1, 3)
```

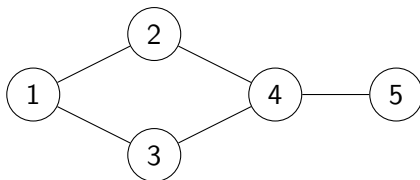
```
add_edge!(G, 2, 4)
```

```
add_edge!(G, 3, 4)
```

```
add_edge!(G, 4, 5)
```

```
#
```

```
G
```



Directed graphs and adjacency lists

```
using Graphs
```

```
# Directed graph with 5 vertices
```

```
G = SimpleDiGraph(5)
```

```
add_edge!(G, 1, 2)
```

```
add_edge!(G, 1, 3)
```

```
add_edge!(G, 2, 4)
```

```
add_edge!(G, 3, 4)
```

```
add_edge!(G, 4, 5)
```

```
println(outneighbors(G, 1)) # vertices
```

```
reachable from 1
```

```
println(inneighbors(G, 4)) # predecessors of 4
```

Important modeling choice

SimpleGraph and SimpleDiGraph store the topology. For a weighted shortest-path problem, we can store weights separately.

Weighted arcs: simple representation

For a small example, a dictionary is very readable:

```
weight = Dict{Tuple{Int,Int},Float64}()  
  
weight[(1, 2)] = 2.0  
weight[(1, 3)] = 5.0  
weight[(2, 3)] = 1.0  
weight[(2, 4)] = 2.0  
weight[(3, 4)] = 1.0  
weight[(4, 5)] = 3.0
```

Then the length of arc (u, v) is:

```
weight[(u, v)]
```

Dijkstra's assumption

All arc lengths must be nonnegative.

Shortest paths: the key idea of Dijkstra

Given a source vertex s , Dijkstra computes tentative distances $d[v]$.

- 1 Start with $d[s] = 0$ and $d[v] = +\infty$ for $v \neq s$.
- 2 Repeatedly choose an unprocessed vertex u with minimum $d[u]$.
- 3 For every arc (u, v) , test whether

$$d[u] + w(u, v) < d[v].$$

- 4 If yes, update $d[v]$ and remember that the predecessor of v is u .

Why a heap?

The operation *extract the smallest tentative distance* is repeated many times. A min-heap makes it efficient.

DataStructures.jl: heaps

BinaryMinHeap maintains the smallest element at the top.

```
using DataStructures
```

```
H = BinaryMinHeap{Tuple{Float64,Int}}{}
```

```
push!(H, (0.0, 1))
```

```
push!(H, (5.0, 3))
```

```
push!(H, (2.0, 2))
```

```
println(first(H))    # (0.0, 1)
```

```
println(pop!(H))     # removes and returns (0.0,
```

1)

Pairs are ordered lexicographically:

$$(d_1, v_1) < (d_2, v_2)$$

first compares the distances d_1, d_2 .

DataStructures.jl: priority queues

A priority queue stores keys and priorities.

```
using DataStructures
```

```
PQ = PriorityQueue{Int,Float64}()
```

```
enqueue!(PQ, 1, 0.0)
```

```
enqueue!(PQ, 3, 5.0)
```

```
enqueue!(PQ, 2, 2.0)
```

```
u, du = dequeue_pair!(PQ)    # key with smallest  
priority
```

For this implementation

We use BinaryMinHeap with pairs (distance, vertex). This avoids a technical decrease-key operation: if a distance improves, we simply insert a new pair.

Dijkstra implementation: initialization

```
using Graphs
using DataStructures

function dijkstra(G::SimpleDiGraph,
weight::Dict{Tuple{Int,Int},Float64},
s::Int)
    n = nv(G)
    # dist[v] = best distance currently known
    from s to v
    dist = fill(Inf, n)
    # pred[v] = predecessor of v in the
    shortest-path tree
    pred = zeros(Int, n)
    dist[s] = 0.0
    # each heap element is (distance, vertex)
    H = BinaryMinHeap{Tuple{Float64,Int}}()
    push!(H, (0.0, s))
```

Dijkstra implementation: main loop

leave to an exercise

Solution of Dijkstra main loop

```
while !isempty(H)
    du, u = pop!(H)

    # ignore an obsolete pair already
    improved later
    if du > dist[u]
        continue
    end

    for v in outneighbors(G, u)
        alt = du + weight[(u, v)]
        if alt < dist[v]
            dist[v] = alt
            pred[v] = u
            push!(H, (alt, v))
        end
    end
end

return dist, pred
```

Reconstructing one shortest path

The array `pred` stores the predecessor of each vertex in the shortest-path tree.

```
function path_to(pred::Vector{Int}, s::Int, t::
Int)
    path = Int[]
    v = t
    while v != 0
        push!(path, v)
        if v == s
            return reverse(path)
        end
        v = pred[v]
    end
    return Int[]           # empty path: t is unreachable
from s
end
```

If `path_to(pred, s, t)` returns an empty vector, there is no directed

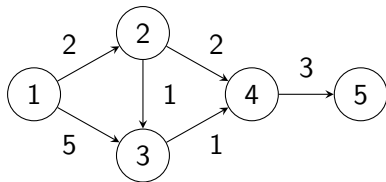
Complete example

```
SimpleDiGraph(5)
edges = [(1,2), (1,3), (2,3),
(2,4), (3,4), (4,5)]
```

```
(u, v) in edges
add_edge!(G, u, v)
```

```
weight = Dict(
(1,2) => 2.0,
(1,3) => 5.0,
```

G =



for

Expected result

Shortest path from 1 to 5:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 5$

with total length 7.

end

Let $n = |V|$ and $m = |E|$.

With a binary heap and a decrease-key operation:

$$O((n + m) \log n).$$

With the duplicate-pairs implementation used here:

- each successful relaxation inserts a new pair in the heap;
- obsolete pairs are ignored when extracted;
- at most m useful relaxations occur;
- the complexity is $O(m \log m)$, which is $O(m \log n)$ for a simple graph.

Comparison with built-in shortest paths

Graphs.jl also provides built-in shortest-path routines.

```
distmx = fill(Inf, nv(G), nv(G))

for ((u, v), c) in weight
    distmx[u, v] = c
end

state = dijkstra_shortest_paths(G, 1, distmx)

println(state.dists)
println(state.parents)
```